



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS**

**A MINOR PROJECT REPORT ON SMART GYM ASSISTANT USING ESP32
AND KNN**

BY

Aniket Nayak (PUR080BCT013)

Bibek Sodari (PUR080BCT025)

Guru Prakash Gupta (PUR080BCT032)

**DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PURWANCHAL CAMPUS
DHARAN, NEPAL**

2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Purwanchal Campus, for providing the facilities and academic environment required to complete this minor project. We are thankful to our project supervisor and faculty members for their guidance, feedback, and technical suggestions throughout the design and implementation process.

We also acknowledge the open-source developers of the ESP32, MPU6050, Arduino, FastAPI, and Python ecosystems. These tools made it possible to develop the sensing, embedded inference, data collection, and monitoring components of the project.

Guru Prakash Gupta

PUR080BCT032

ABSTRACT

This minor project presents a Smart Gym Assistant for monitoring exercise movement and posture using an ESP32, two MPU6050 inertial measurement units, an embedded K-nearest neighbors (KNN) model, and a FastAPI web service. The two sensors provide acceleration and gyroscope measurements from different body locations, producing twelve motion features for each sample. The ESP32 collects the readings at approximately 50 Hz, classifies the movement locally, displays the result on an OLED screen, and transmits sensor batches through Wi-Fi.

A Python training script standardizes the sensor features and generates a compact KNN model header for deployment on the ESP32. The current model distinguishes bicep movement states labelled `bicep_bad`, `bicep_good`, and `bicep_idle_good`. The FastAPI application receives labeled data, stores it in CSV format for later training, accepts live predictions, and provides a browser-based monitoring interface. Double buffering on the ESP32 allows sensor collection to continue while another batch is being uploaded.

The system demonstrates a practical edge-AI architecture in which time-sensitive prediction is performed on the microcontroller while data storage and visualization are handled by a lightweight backend. It can be extended with additional exercises, more users, repetition counting, and a database-backed analytics system.

Keywords: ESP32, MPU6050, wearable sensing, KNN, posture detection, FastAPI, edge machine learning

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Background	1
1.2 Gap Identification	1
1.3 Motivation	2
1.4 Objectives	2
1.5 Scope	2
2 RELATED THEORY	3
2.1 Inertial Measurement Unit	3
2.2 Feature Representation	3
2.3 K-Nearest Neighbors	3
2.4 Edge Computing and REST Communication	3
3 LITERATURE REVIEW	5
4 METHODOLOGY	6
4.1 Overview	6
4.2 Hardware Design	6
4.3 Data Collection and Preparation	6

4.4	Model Training and Deployment	6
4.5	FastAPI Integration	7
4.6	Testing Procedure	7
5	EXPECTED RESULTS	8
6	CONCLUSION	9
	REFERENCES	10
	APPENDIX	11

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

API	: Application Programming Interface
ESP32	: Espressif 32-bit microcontroller
I2C	: Inter-Integrated Circuit
IMU	: Inertial Measurement Unit
KNN	: K-Nearest Neighbors
MPU	: Motion Processing Unit
REST	: Representational State Transfer
CSV	: Comma-Separated Values

CHAPTER 1

INTRODUCTION

1.1 Background

Exercise quality is affected by posture, movement range, and consistency. In a gym environment, beginners may perform an exercise incorrectly without receiving immediate feedback. Human supervision can address this problem, but it is not always available. A wearable sensing system can provide a low-cost alternative by measuring body motion and identifying movement patterns.

This project develops a Smart Gym Assistant using an ESP32 and two MPU6050 sensors. Each MPU6050 provides three-axis acceleration and three-axis gyroscope data. The combined twelve-dimensional sensor vector is classified by a K-nearest neighbors (KNN) model deployed directly on the ESP32. A FastAPI application receives sensor data and live predictions over Wi-Fi and displays the latest exercise state in a browser dashboard.

1.2 Gap Identification

Many exercise-monitoring solutions depend on camera-based pose estimation or continuous cloud processing. Camera systems can raise privacy concerns and may be affected by lighting and occlusion. Cloud-based systems also add network delay and require a reliable connection for every prediction. There is a need for a compact system that can perform basic classification locally while still providing a convenient way to collect data and monitor results.

The proposed system addresses this gap by combining wearable inertial sensing with edge inference. The ESP32 performs the prediction locally, while the server is used for data collection, visualization, and future model improvement. The current prototype is focused on bicep movement states and is designed to be extensible to other exercises.

1.3 Motivation

The motivation of this project is to make exercise feedback more accessible using inexpensive hardware and an interpretable machine-learning method. KNN is suitable for the prototype because it is simple to train, can work with a small labeled dataset, and can be embedded as a compact collection of prototypes. Using two sensors also provides motion information from more than one body location, which can improve discrimination between similar movements.

1.4 Objectives

- To collect acceleration and gyroscope data using two MPU6050 sensors.
- To classify bicep movement and posture using an embedded KNN model.
- To provide immediate prediction feedback through an OLED display.
- To transmit sensor batches and predictions from the ESP32 to a FastAPI server.
- To store labeled sensor data for future training and evaluation.
- To provide a live browser-based monitoring interface.

1.5 Scope

The scope of the current prototype includes dual-sensor data acquisition, local classification of three bicep movement states, Wi-Fi communication, CSV data storage, and live status visualization. Medical diagnosis, complete athletic assessment, and guaranteed injury prevention are outside the scope of this project.

CHAPTER 2

RELATED THEORY

2.1 Inertial Measurement Unit

An inertial measurement unit measures motion using accelerometers and gyroscopes. An accelerometer measures specific force along the three axes, while a gyroscope measures angular velocity. The MPU6050 combines both sensing functions in one device and communicates with the ESP32 through the I2C bus [1]. Two devices can share the same bus when they use different I2C addresses, in this project 0×68 and 0×69 .

2.2 Feature Representation

For every sample, the system records six values from each sensor:

$$\mathbf{x} = [a_{x1}, a_{y1}, a_{z1}, g_{x1}, g_{y1}, g_{z1}, a_{x2}, a_{y2}, a_{z2}, g_{x2}, g_{y2}, g_{z2}].$$

The twelve values form the input feature vector. Since acceleration and angular velocity have different numerical ranges, the training script standardizes each feature using its mean and scale before calculating distances.

2.3 K-Nearest Neighbors

KNN classifies an unknown sample by finding the k training samples with the smallest distance. The class receiving the majority of votes is returned as the prediction [2]. This project uses $k = 5$ and Euclidean distance over standardized features. The trained prototypes, feature means, feature scales, and labels are generated as a C++ header and stored in ESP32 program memory.

2.4 Edge Computing and REST Communication

Edge computing performs processing near the data source. In this project, the ESP32 performs prediction locally so that feedback does not depend on a server response [3].

The FastAPI application provides REST endpoints for receiving sensor batches and live predictions [4]. The browser dashboard requests the latest status and marks the ESP32 offline when no recent update has been received.

CHAPTER 3

LITERATURE REVIEW

Wearable exercise-monitoring systems commonly use inertial sensors because they are inexpensive, portable, and less sensitive to lighting and camera occlusion than vision-based systems. Accelerometers are useful for detecting changes in linear motion, while gyroscopes provide information about rotation. Combining both measurements gives a richer description of exercise movement.

Machine-learning methods have also been applied to activity and posture recognition. KNN is an attractive method for small embedded prototypes because training is straightforward and the classification rule is easy to interpret. Its main limitations are the need to retain representative samples and the effect of feature scaling on distance calculations. These limitations are addressed here by selecting a bounded number of prototypes per label and standardizing every feature.

Cloud and web-connected fitness systems simplify storage and visualization, but network dependence can introduce delay and privacy concerns. The architecture in this project separates responsibilities: the ESP32 performs immediate classification, and the FastAPI service stores data and exposes monitoring functions. This approach combines the responsiveness of edge processing with the convenience of a web interface.

The reviewed approaches motivate a dual-MPU6050, edge-KNN design for the prototype. Future evaluation should compare KNN with other classifiers using a separate test set and should measure accuracy across different users and sensor placements.

CHAPTER 4

METHODOLOGY

4.1 Overview

The system is implemented in four stages: sensor acquisition, model training, embedded inference, and server-side monitoring. The ESP32 reads both sensors at a fixed interval and places samples into alternating memory buffers. When one buffer is full, it is handed to the upload task while the collector continues using the other buffer.

4.2 Hardware Design

The two MPU6050 modules are connected to the ESP32 through I2C. Their AD0 pins are configured so that the first sensor uses address `0x68` and the second uses address `0x69`. An SH1106 OLED is connected to display collection state, sample count, prediction, posture, and confidence. The ESP32 uses Wi-Fi to communicate with the computer running FastAPI.

4.3 Data Collection and Preparation

Each sample contains a timestamp and twelve sensor values. During collection, the sample is tagged with the exercise, posture, and user name. The FastAPI endpoint validates the JSON structure and appends valid batches to `sensor_data_labeled.csv`. The Python training script reads the labeled CSV, groups rows by label, selects a maximum of 64 evenly distributed prototypes per label, and calculates the mean and standard deviation for each feature.

4.4 Model Training and Deployment

The standardized prototypes and labels are written to `knn_model.h`. During inference, the ESP32 standardizes the incoming twelve-feature vector using the stored means and scales. It calculates the squared Euclidean distance to each embedded prototype, keeps the five nearest prototypes, and returns the majority label. The current deployed

labels are `bicep_bad`, `bicep_good`, and `bicep_idle_good`.

4.5 FastAPI Integration

The server exposes an endpoint for batched sensor data and another endpoint for live prediction updates. Collection-mode batches are stored in CSV format, while live prediction messages update an in-memory status object. The dashboard reads this status and reports the exercise, posture, confidence, number of samples, and online state.

4.6 Testing Procedure

Testing consists of checking sensor initialization, confirming that both I2C addresses respond, verifying KNN label output for recorded samples, checking JSON transmission from the ESP32, and confirming that the server stores valid collection batches. Model evaluation should use a separate test portion of the data; the generator's prototype self-check alone is not a sufficient estimate of generalization accuracy.

CHAPTER 5

EXPECTED RESULTS

The completed prototype is expected to produce the following results:

- Both MPU6050 sensors initialize successfully using their separate I2C addresses.
- The ESP32 collects motion samples at approximately 50 Hz.
- The embedded KNN model returns one of the three deployed bicep movement labels.
- The OLED displays the current collection state and latest prediction.
- The FastAPI server receives valid batches over Wi-Fi without corrupting CSV rows.
- The browser dashboard displays the latest exercise, posture, confidence, and connection state.
- Recorded data can be reused to regenerate the embedded model after additional labeling.

The final accuracy should be reported from an independent test set collected under different repetitions and, where possible, from more than one user. A numerical accuracy value is intentionally not claimed here until that test is performed.

CHAPTER 6

CONCLUSION

The Smart Gym Assistant combines wearable motion sensing, embedded machine learning, wireless communication, and a web interface into one exercise-monitoring system. Two MPU6050 sensors connected to the ESP32 provide synchronized acceleration and gyroscope data. The embedded KNN classifier uses the twelve sensor values to identify the current bicep movement state, while the OLED provides immediate local feedback.

The FastAPI service complements the embedded system by receiving sensor batches, storing labeled samples, accepting live predictions, and presenting the latest status through a browser dashboard. The use of double buffering helps separate fixed-rate data collection from comparatively slower network transmission. This makes the design suitable for experiments in real-time posture monitoring without requiring continuous cloud processing.

The present implementation is a working prototype. Its deployed model contains three bicep-related labels, while the available data collection interface can be used to collect additional exercise classes. The accuracy of the final system depends on sensor placement, user variation, dataset balance, and the quality of the recorded labels. Future work should therefore include a larger and more diverse dataset, independent test evaluation, repetition counting, and additional exercises such as squats and push-ups.

Overall, the project demonstrates how low-cost inertial sensors and edge machine learning can support practical fitness-assistance applications. It provides a foundation for a more complete coaching system that can give immediate posture feedback while retaining sensor data for later analysis and model improvement.

REFERENCES

- [1] InvenSense, *MPU-6000 and MPU-6050 Product Specification*, 2013.
- [2] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [3] Espressif Systems, *ESP32 Series Datasheet*, 2024.
- [4] S. Ramírez, “Fastapi documentation,” 2024. [Online]. Available: <https://fastapi.tiangolo.com/>

APPENDIX A

A.1 Software and Model Files

The project source is organized into the following files:

- `esp32_dual_mpu6050_post.ino`: sensor acquisition, buffering, OLED display, and HTTP upload.
- `model.ino`: adapter functions for embedded KNN prediction.
- `train_knn.py`: CSV loading, feature scaling, prototype selection, and header generation.
- `knn_model.h`: generated model data and prediction function.
- `fastapi_api/app/main.py`: data receiver, live prediction API, and CSV storage.

APPENDIX B

B.1 API Endpoints

Endpoint	Purpose
POST /api/sensor-data	Receive sensor batches and store collection data
GET /api/live-status	Return the latest prediction and device state
GET /api/live-prediction	Receive a live ESP32 prediction
GET /api/sensor-data/summary	Return stored row counts by label